

Szakképesítés neve: Szoftverfejlesztő és -tesztelő
OKJ száma: 54 522

Vigrid

Készítette:

Simon Attila Máté
Kertmegi Richárd Imre
Medve Márk

A Játék

Bevezető

A *Vigrid* egy videójáték, amelyet a Slay the Spire által inspirált mechanikák, valamint az északi mitológia gazdag világának ötvözésével fejlesztettünk. A projekt célja egy olyan stratégiai alapokon nyugvó, kártya alapú harc rendszerrel rendelkező játék létrehozása, amely könnyed kikapcsolódást nyújthat az egyszerűbb stratégiai alapokon nyugvó szoftverekre vágyóknak. A projekt feladatként bemutatott demo egyenlőre egy karaktert, a bondit helyezi előtérbe, annak kihasználását a játékosra bízva. Azért választottuk ezt a témát és zsánert, mert mind a pakli építős rouglike játékok, mind az északi mitológiára alapuló történetek nem kapják meg a megfelelő figyelmet, amit érdemelnének.

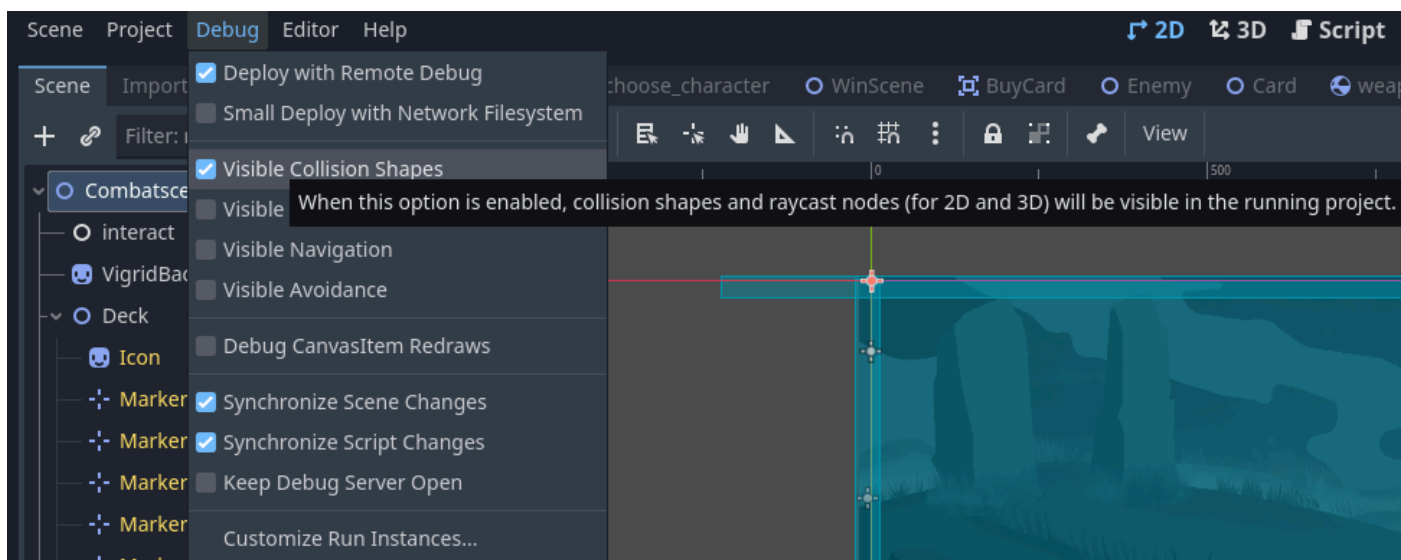
A program célja, hogy:

- egyedi és újrajátszható játékelményt nyújtson
- fejlessze a játékos stratégiai gondolkodását és döntéshozatali képességeit
- bemutassa a kártyaalapú játékmechanikák hatékony implementációját

Tesztelés

A programot a Godot játékmotor beépített debug felületével teszteltük, minimalizálva a futás közben előforduló hibákat és teljesítmény belüli ugrásokat. Annak érdekében, hogy a tesztek során a program minél kevesebbet sérüljön, sok tesztet más Godot projektekben végeztünk el, és aztán importáltuk át az ott kapott végleges változatot a főprogramba. Így készült el a map, a menü és a harcrendszer is.

A Godot játékmotor beépített debug felülete lehetővé teszi a programozó számára, hogy könnyen futtassa a programot tesztelői szemmel, ahol a játékban fellelhető háttér mechanikák láthatóvá válnak.

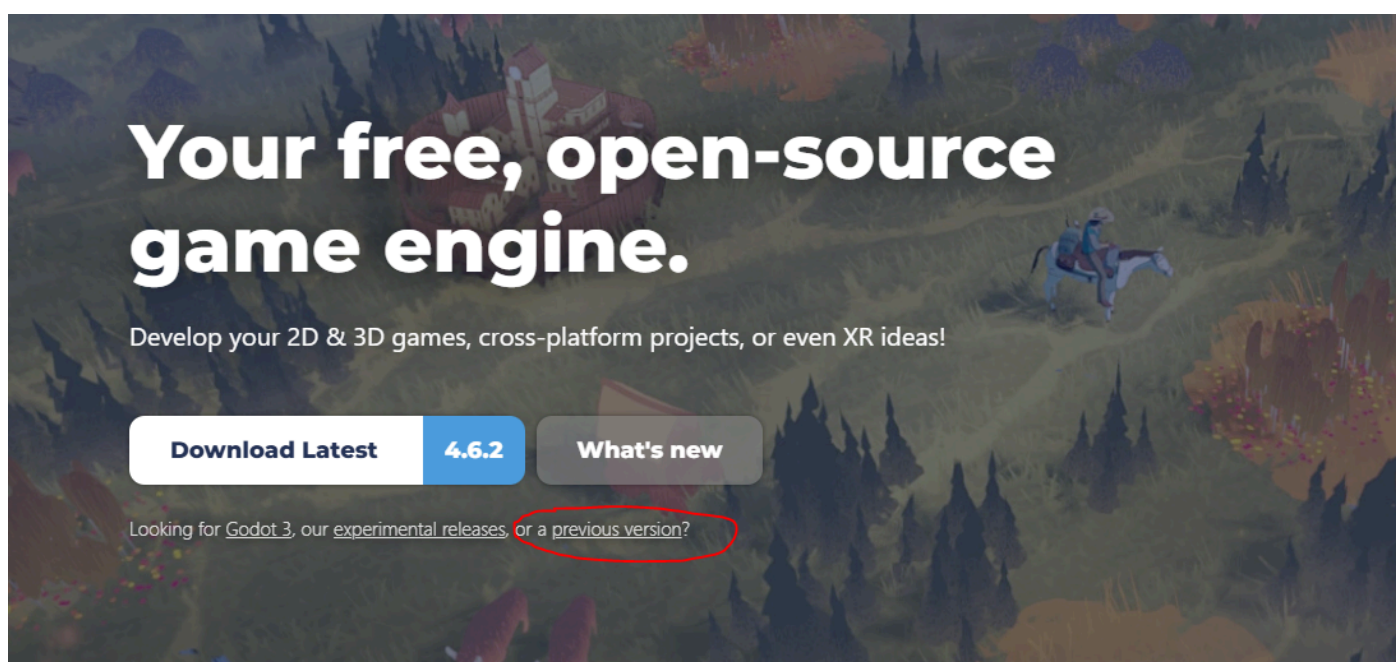


Letöltés és elindítás

A program letöltéséhez szükség van annak Github linkjére. Az onnan letöltött mappa tartalmazza a Vgrid.exe fájlt egy tömörített mappában. Azt kicsomagolva futtatható a játék az exe fájlon keresztül

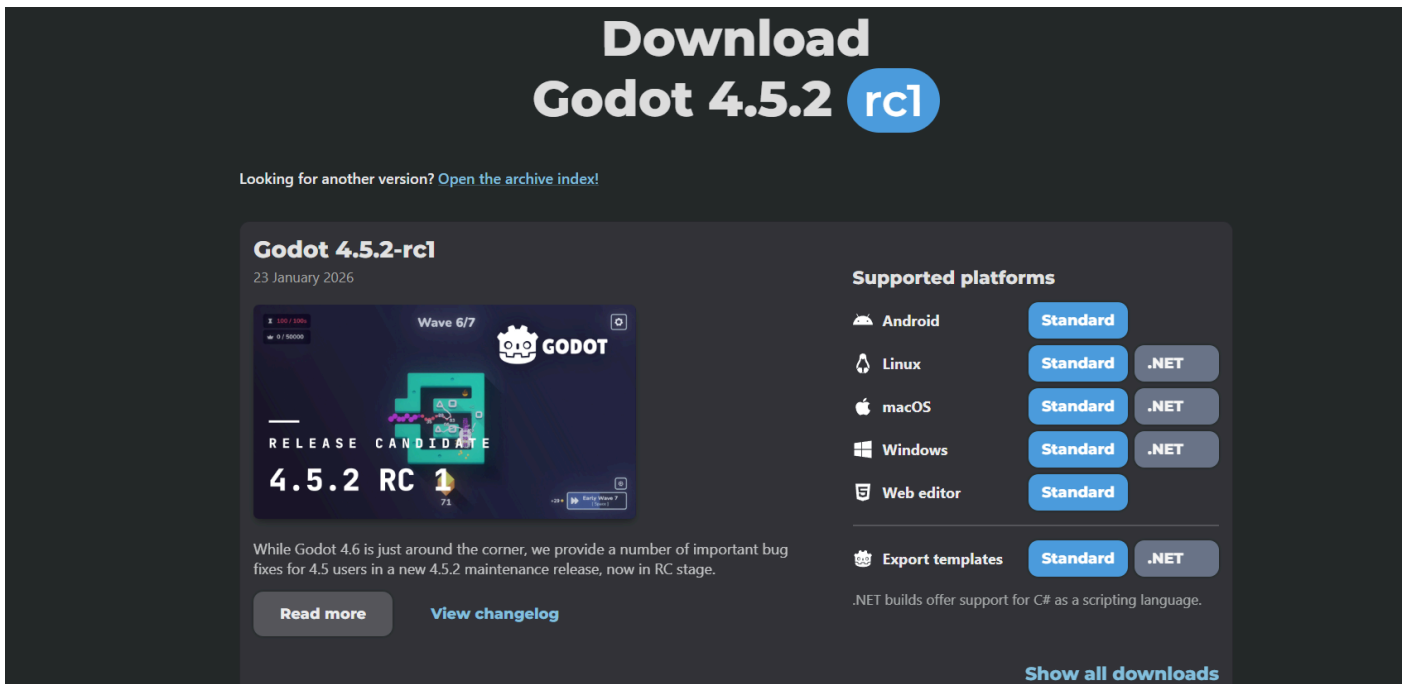
Ahhoz, hogy a Godot felületén megnyitható legyen a program, szükség van a Godot program letöltésére, amit a Godot hivatalos oldaláról tehetünk meg.

A link: <https://godotengine.org>

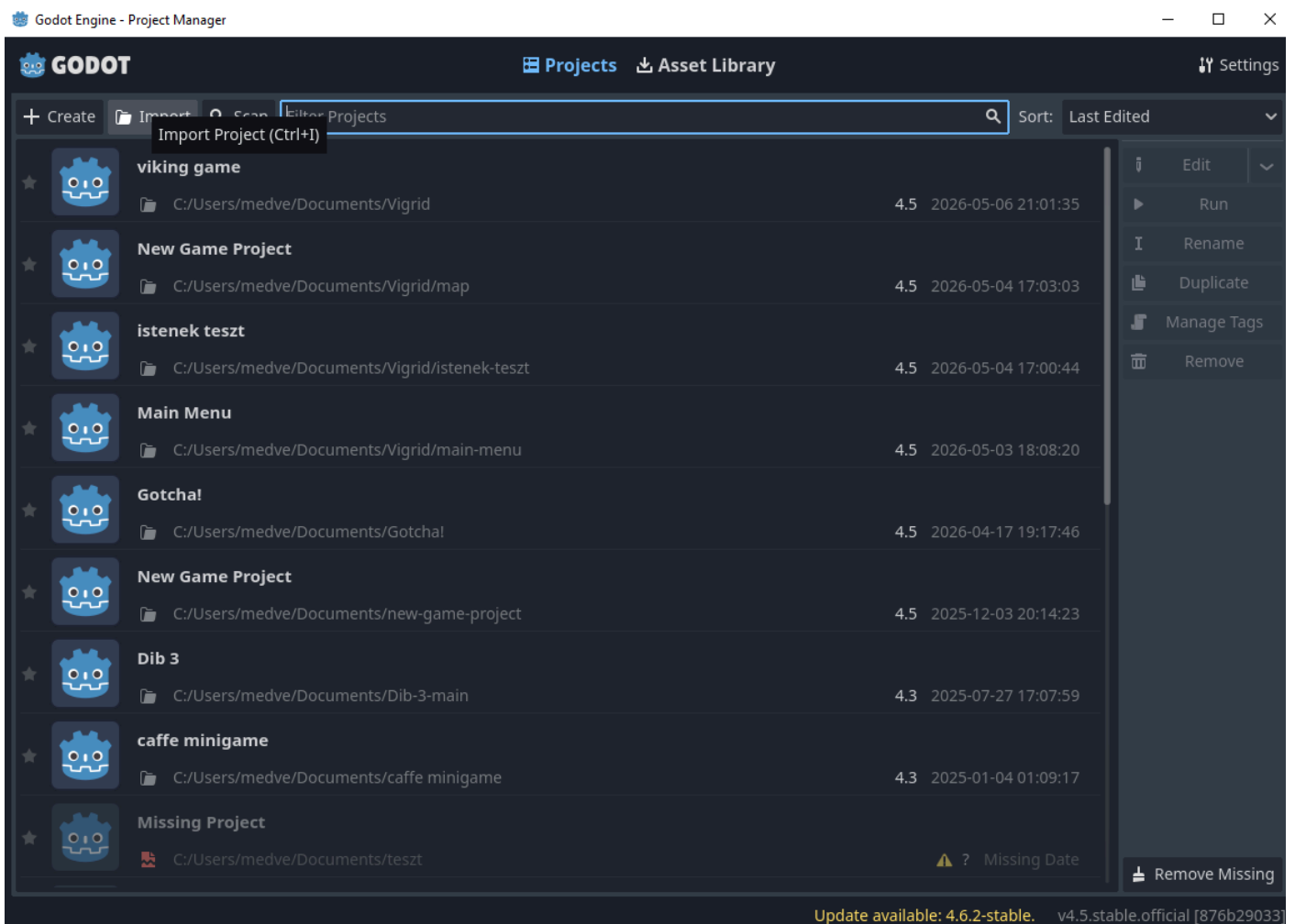


A játék a Godot egy előző verziójában (4.5.2) készült, annak letöltéséhez a “previous version” gombra kattintva juthatunk el.

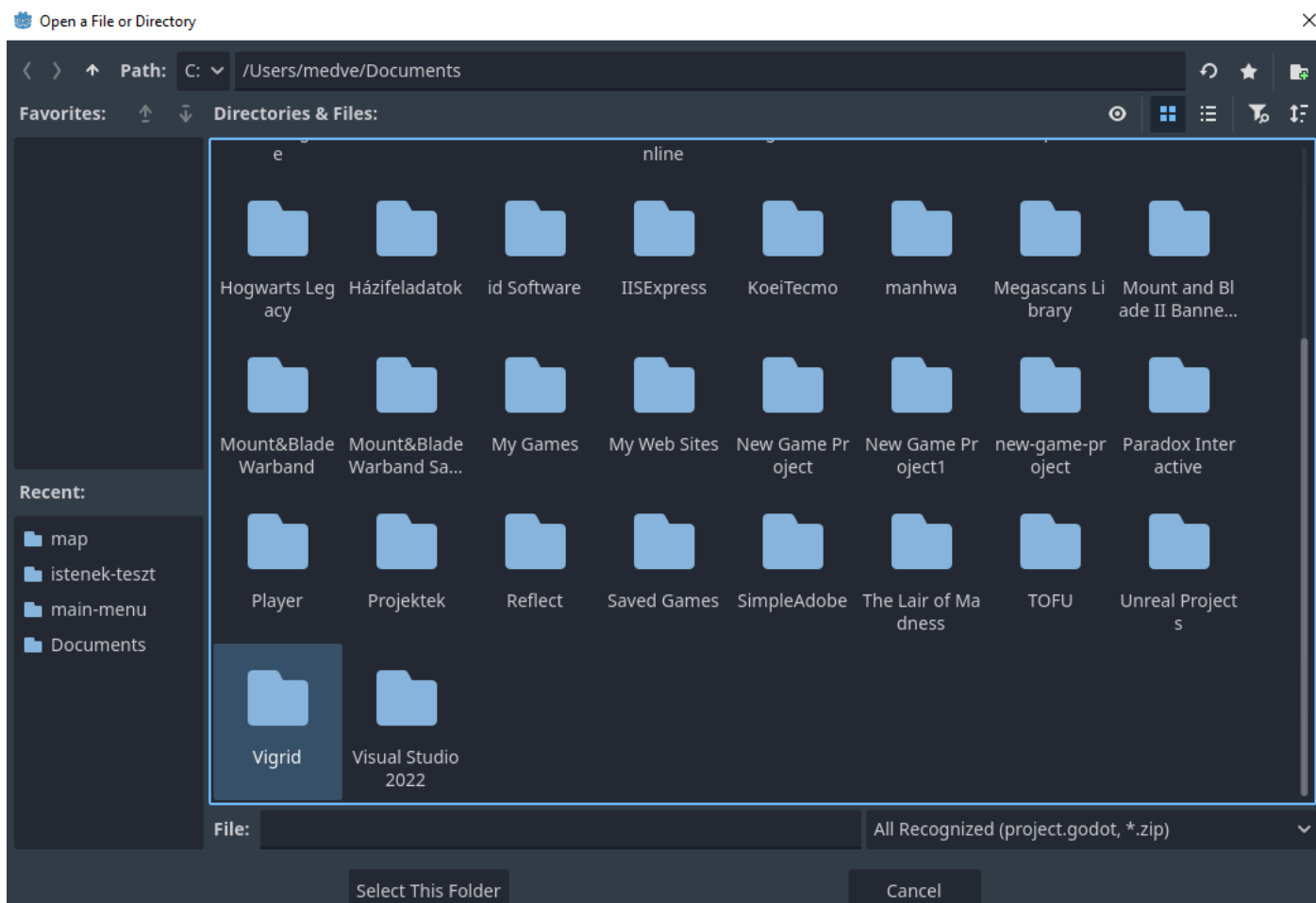
Itt letekerünk, ameddig a kívánt verziószámmal nem találkozunk. Az itt felvonultatott lehetőségek közül (stable vagy rc1) bármelyik megfelel a program futtatására.

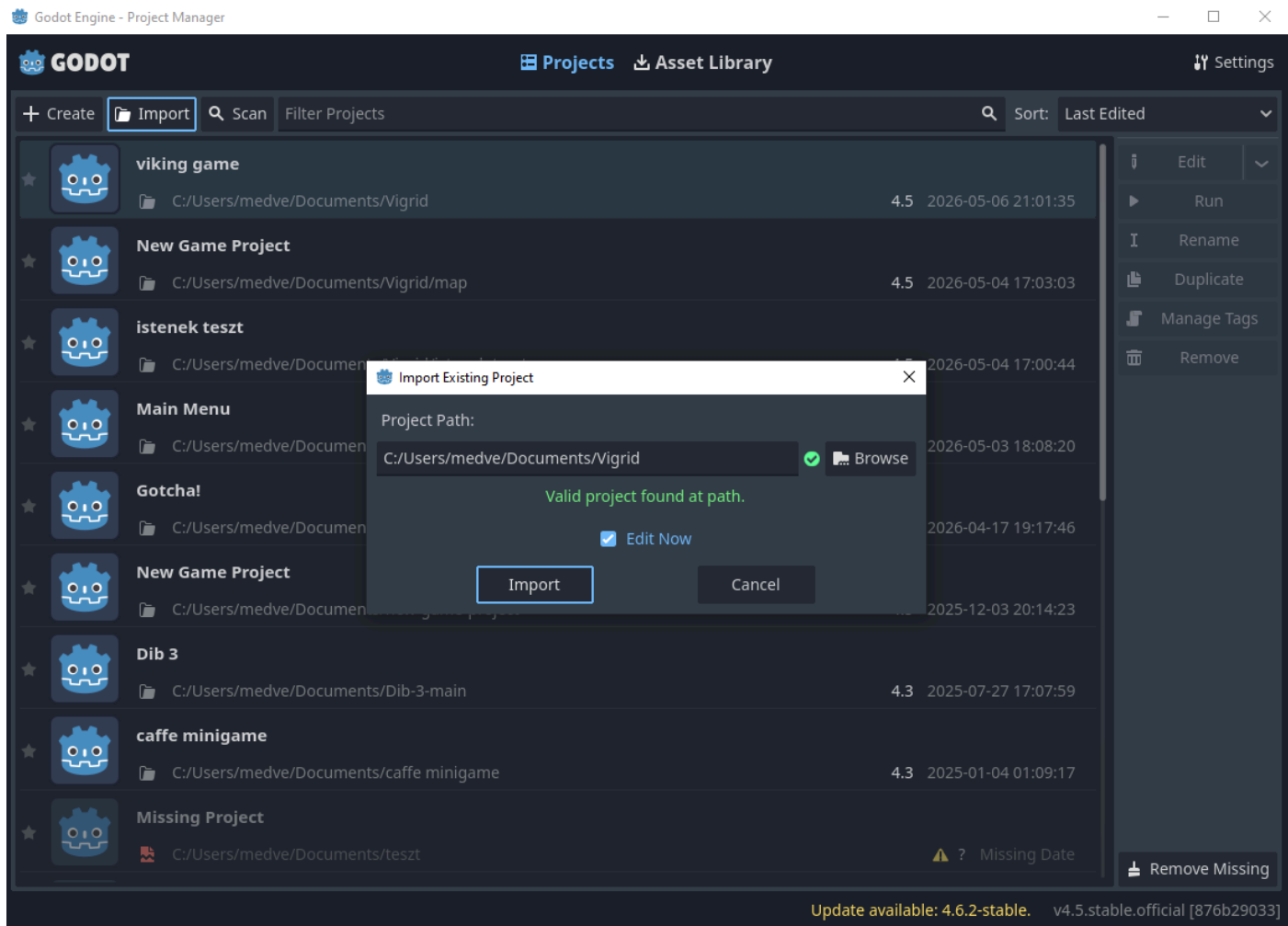


Ha bármelyik választható lehetőségnél rákattintunk a “download” gombra, a weboldal átdob minket a képen látható lapra, ahol kiválasztva a “Windows” opciót, a program alacsony gépigényének köszönhetően percekben belül elindíthatjuk azt, nincs szükség lokális telepítésre.



A program megnyitása után a képen látható felület jelenik meg. Itt az Importálás gombra kattintva kikereshetjük a Vigrid mappáját, amit előzőleg a Githubról letöltöttünk. A Godot automatikusan a Dokumentumok közt kezdi el keresni a kívánt mappát, ám ha máshova mentettük, a felső navigációs gombok, vagy a mappa helyének manuális beírásával könnyen megtalálható a program, amit megnyitni szeretnénk.





A Játék

1. A játék menete:

A program elindítása utáni első képernyő, a menü. Itt, a rúnákkal jelölt gombok segítségével haladhat tovább a felhasználó, valamint a csontra rákattintva juthat el az opciók menüpontra. Ez a vizuális döntés annak érdekében született, hogy a játékban minél kevesebb hangulat öltő elem kerüljön. Minden menüpont egy rúna, a játék címe egy kardba van vésve.



Az új játék, vagy “New Game” gombra kattintva a program átvált a karakterválasztó oldalra. Itt jelenleg egy karakter elérhető (Bondi), illetve egy további tesztelést igényel. Ha a felhasználó bármelyik menüpont fölé viszi az egerét, az világítani fog, illetve kiírja az adott menüpont célját (Úgy mint: New Game, Options, Continue, Exit)

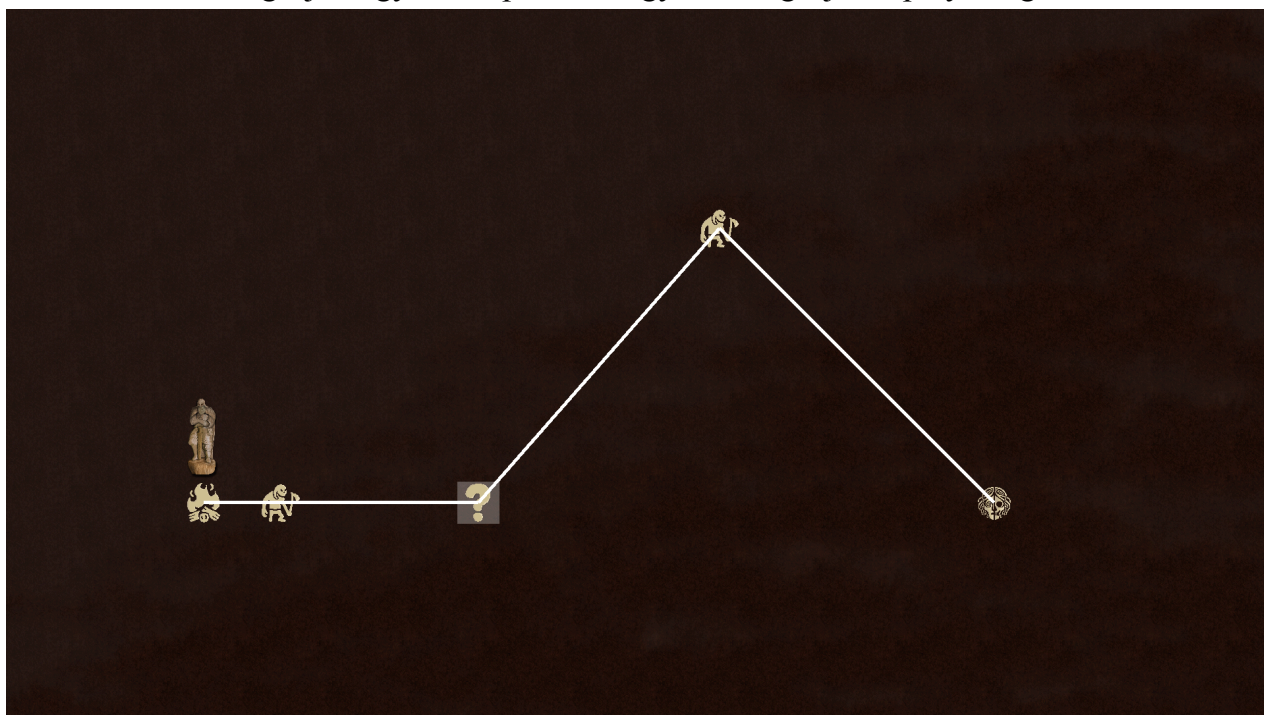




Ellenfél



Pálya végi főellenség (Jelenleg Hel)





A játékosnak itt a karaktertől függő mennyiségű energiája (A baloldalt látható szám) és előre megadott paklija van, ami később bővíthető. A kártyák a bal felső sarkukban jelzett energia mennyiségért cserébe hívhatók elő.

A kártyák előhívásához a kívánt kártyára kell kattintani, majd a képernyő közepére húzni. Ha nincs elég energiája a karakternek ahhoz, hogy a kártya aktiválva legyen, akkor a kártya nem fog mozdulni a helyéről. Optimális esetben (Ha a kártya előhívásához van elég energiánk), a kártya mozdítható, a képernyő közepére húzva pedig apró fehér csillogás fogja jelezni, hogy aktiválható.



Az egér elengedésével a kártya aktiválódik, annak ára levonásra kerül az energiából, és függően a kártya céljától, vagy szimplán eltűnik, vagy a helyén egy fizika alapján működő tárgy jelenik meg.

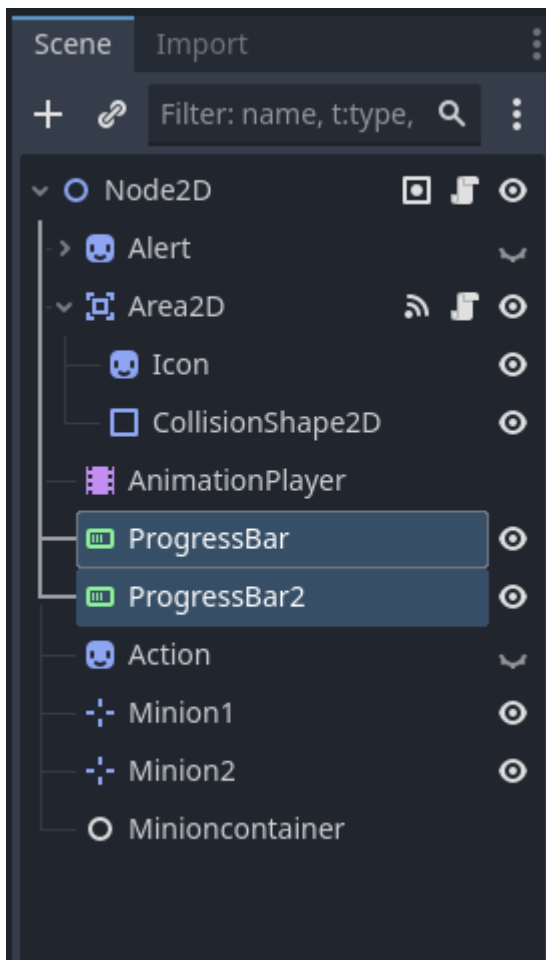


Bármely karakterre dobva a tárgyat, annak hatása aktiválódik (pajzs esetén az “áldozat” +3 pajzsot nyer, bármiféle támadás esetén a fegyver előre deklarált sebzési pontja levonásra kerül az áldozat életerejéből/pajzsából)

2. A játék működése.

1. A motor

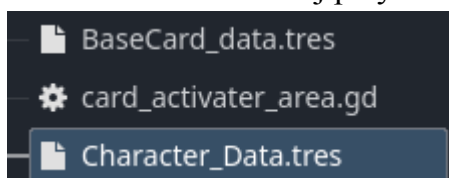
A Godot játékmotor scene-ekre és scriptekre épül, amiket a saját program nyelvével, a GDScriptel használhatunk, valamint lehetőség van C++ kódot is használni benne. A scene-ek komponens alapú, hierarchikus entitások. A struktúrája fa alapú, a motorban használt kifejezés egy projektre ezért is tree.



2. Adatbázisok és betöltések

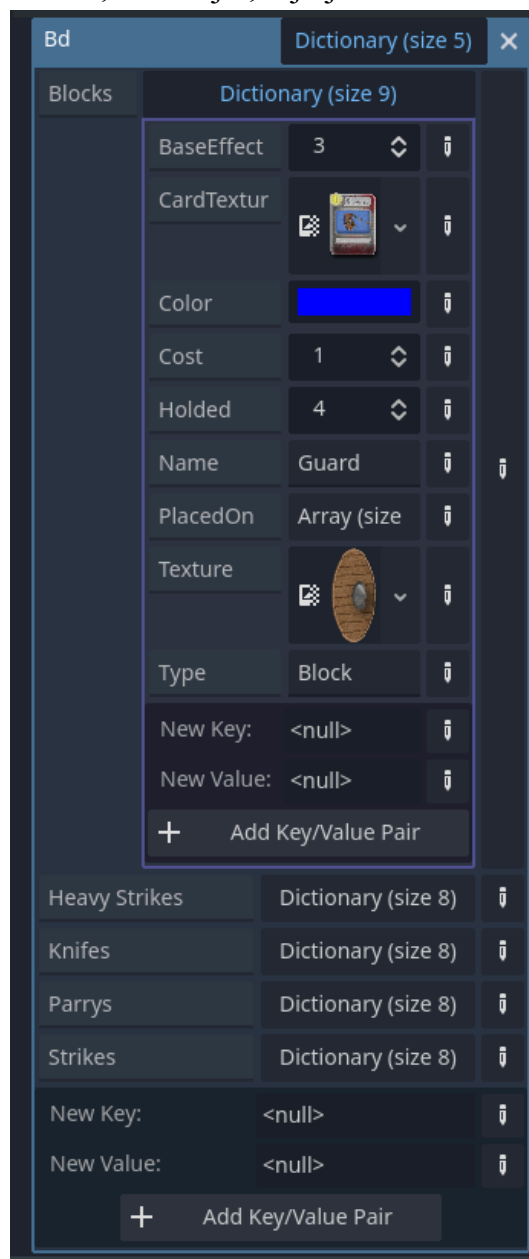
A Vigrid két saját adatbázissal operál, ezek a BaseCard_data.tres, illetve a Character_Data.tres.

Ez a két adathordozó fájl felelős a kártyák eltárolásáért, illetve a karakter és az ellenfelek létrehozásáért minden új pályán.



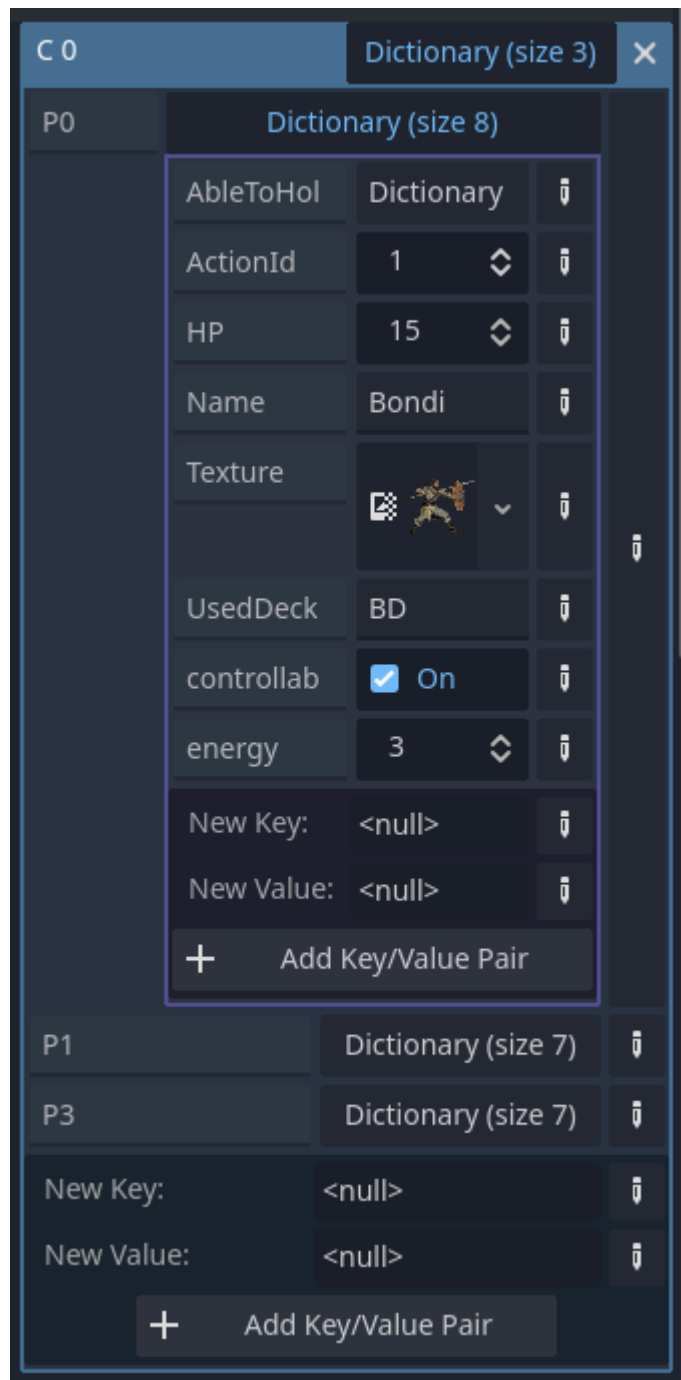
1. BaseCard_data.tres

A játékos alap pakliját tárolja, a kód innen képes lehívni a létrehozni kívánt kártya nevét, alap értékét, textúráját, fajtáját.



2. Character_data.tres

A játékos karakter és az ellenfeleket közösen tárolja a megegyező értékeik miatt, valamint a könnyű hozzáférhetőség érdekében. Az adatbázis tartalmazza a karakter ID-ját, élet pontját, nevét, textúráját, használható paklijának egyedi azonosítóját, illetve, hogy mennyi energiával kezd.



P0	Dictionary (size 7)		
ActionId	0	↕	ⓘ
Actions	Dictionary		ⓘ
HP	20	↕	ⓘ
Name	Wolf		ⓘ
Texture	 ▾		ⓘ
controllab	☐ On		ⓘ
energy	1	↕	ⓘ
New Key:	<null>		ⓘ
New Value:	<null>		ⓘ
+ Add Key/Value Pair			

Az ellenfél egy leegyszerűsített változatként jelenik meg, a használható akcióin, nevén, textúráján túl nincs más, ami szükséges lenne a létrehozásához.

3. A Szintaktika

A karakterek megjelenítéséhez először is fontos, hogy a `Character_data.tres`-ből kapott adatokat külön válogassuk: Mi-kinek szól?

```
func create():
    >I var choosedcharacter
    >I var basecharacter = CHARACTER_DATA.get_meta("C"+str(sid))
    >I if sid == 0:
        >I >I choosedcharacter = basecharacter[MouseState.Character]
        >I >I controllabel = choosedcharacter.controllabel
        >I >I Interact.playercharacters.append(sid)
        >I >I MouseState.Energy = choosedcharacter.energy
        >I >I MouseState.choseddeck = choosedcharacter.UsedDeck
    >I else:
        >I >I choosedcharacter = basecharacter[MouseState.ChoosedEnemy]
        >I >I controllabel = choosedcharacter.controllabel
    >I actionid = choosedcharacter.ActionId
    >I maxhp = choosedcharacter.HP
    >I hp = choosedcharacter.HP
    >I texture = choosedcharacter.Texture
    >I icon.texture = texture
```

A kód egy egyszerű if-else ággal külön válassza bekapott információkat. Ha a folyamat először fut le, a `sid=0`, akkor a játékos karakter kerül létrehozásra, bármilyen egyéb esetben az ellenfél.

A kártyák esetében a helyzet igen hasonló, ám hozzá tartozik még némi véletlenszerű keverés is.

```

func _ready():
    indeck.append_array(MouseState.addedtodeck)
    await get_tree().create_timer(0.001).timeout
    basedeck = BaseCard_data.get_meta(combatscene_main.Deck)
    for item in basedeck:
        for x in basedeck[item].Hолded:
            indeck.append(basedeck[item].Name)
    deckshuffel()

```

Ez a kód a harc elindításának pillanatában aktiválódik, kibontja a kártyák adatát tároló metadatát, és elindít egy for ciklust, ami az adattárolóban található kártyák neveit hozzáadja egy tömbhöz.

```

func deckshuffel():
    if indeck.size()>=5:
        cardplace = [marker_2d.global_position, marker_2d_2.global_position, marker_2d_3.global_position, marker_
        for x in 5:
            random = randi_range(0, indeck.size()-1)
            infront.append(indeck[random])
            deckhandler(indeck[random], x)
            indeck.remove_at(random)
    else:
        for i in usedpile.size():
            indeck.append(usedpile[i-1])
        usedpile.clear()
        deckshuffel()

```

Ez után a deckshuffel() függvény egyenként elhelyezi a még üres kártyákat a képernyőn, ügyelve arra, hogy ha esetlegesen a használható pakliban kevesebb mint 5 darab kártya van, úgy a már használtból hozzon vissza eleget.

```
func deckhandler(Name: String, id: int ):
>|  for item in basedeck:
>|  >|  if basedeck[item].Name == Name:
>|  >|  >|  var strike = STRIKE.instantiate()
>|  >|  >|  hand.add_child(strike)
>|  >|  >|  strike.id = id
>|  >|  >|  strike.global_position = cardplace[id]
>|  >|  >|  strike.Type = basedeck[item].Type
>|  >|  >|  strike.Baseeffectnumb = basedeck[item].BaseEffectNumb
>|  >|  >|  strike.Name = basedeck[item].Name
>|  >|  >|  strike.cost = basedeck[item].Cost
>|  >|  >|  strike.cardtexture = basedeck[item].CardTexture
>|  >|  >|  strike.texture = basedeck[item].Texture
>|  >|  >|  if basedeck[item].Name == "Block":
>|  >|  >|  >|  strike.placedon = basedeck[item].PlacedOn
```

A `deckhandler()` függvény felelős azért, hogy a már előzőleg létrehozott, de üres kártyák megkapják a textúrájukat, értékeiket, nevüket.

Egy-egy kártya feláldozása után a kapott fegyvert forgatni és dobálni lehet. Ez a Godotba épített fizikai rendszernek köszönhető.

```
func _physics_process(delta: float) -> void:
>|  if grabbed:
>|    >|  var down = Vector2.DOWN
>|    >|  var tip_direction = transform.y.normalized()
>|
>|    >|  var angle_diff = tip_direction.angle_to(down)
>|
>|    >|  var torque = angle_diff * 200
>|    >|
>|    >|  var target = get_global_mouse_position()
>|    >|  var grab_global = to_global(grab_offset)
>|    >|
>|    >|  var error = target - grab_global
>|    >|
>|    >|
>|    >|  var r = grab_global - global_position
>|    >|  var tangential_velocity = Vector2(-angular_velocity * r.y, angular_velocity * r.x)
>|    >|  var point_velocity = linear_velocity + tangential_velocity
>|    >|
>|    >|
>|    >|  var force = error * 50 - point_velocity * 10
```

A *physics_process* lehetővé teszi, hogy fizikát szimuláljunk a programon belül, a képen lévő kód a fegyverek súlyát és mozgását változtatja, függően attól, hogy hol kattint a játékos rá.

Miután egy csata lezajlott a játékos visszakerül a térképre, ahol ugorhat a következő csatába. A térkép figyeli, hogy hol vagyunk jelenleg, az adott “meghódított” pont ID-ja, és az annál kisebb ID-val rendelkező helyek már nem látogathatóak. Az utána következők közül is csak az 1-el nagyobb.

Maga a térkép úgy lett felépítve, hogy egyszerű, véletlenszerű, de szükség esetén könnyen módosítható legyen.

```
func generateMappoints() -> void:
>| for x in x_cord.size():
>| >| var random = randi_range(0,y_cord.size()-1)
>| >| var map_point = MAP_POINT.instantiate()
>| >| add_child(map_point)
>| >| map_point.global_position = Vector2(x_cord[x],y_cord[random])
>| >| map_point.areajelolo_1 = x
>| >| map[x] = map_point
>| >| if x == 2:
>| >| >| map_point.fight_icon.texture = load("res://map_folder/Random_Icon.png")
>| >| >| var randomevent= randomeventlist.pick_random()
>| >| >| map_point.mapevent.Event.Description = randomevent
>| >| >| map_point.checkdictionary("Event")
>| >| >| randomeventlist.erase(randomevent)
>| >| if x == 4:
>| >| >| map_point.fight_icon.texture = load("res://map_folder/Hel_Icon.png")
>| >| >| map_point.checkdictionary("P4")
```

A Weboldal

1. Követelmények

1.1. Webböngésző (az alábbiak legfrissebb verziója):

- 1.1.1. Google Chrome 90+
- 1.1.2. Mozilla Firefox 88+
- 1.1.3. Safari 14+
- 1.1.4. Microsoft Edge 90+
- 1.1.5. Opera 76+

1.2. Egyéb követelmények:

- 1.2.1. Stabil internetkapcsolat
- 1.2.2. Frissített böngésző, amely támogatja a modern CSS tulajdonságokat és funkciókat

2. Telepítés

2.1. Mivel az a **Vigrid** játék tudástára egy web alapú alkalmazás, a végfelhasználóknak nem szükséges telepíteniük semmilyen szoftvert. Az alkalmazás használatához elegendő egy kompatibilis webböngészővel rendelkező eszköz és internetkapcsolat. A fejlesztői környezet telepítéséhez és a lokális futtatáshoz az alábbi lépések szükségesek:

2.2. 1. Előfeltételek telepítése:

- 2.2.1. Node.js (16.x vagy újabb verzió)
- 2.2.2. npm (Node Package Manager)
- 2.2.3. Git verziókezelő rendszer (Sourcetree vagy GitHub Desktop)

2.3. **Repository klónozása:**

2.3.1. <https://github.com/medvemark-10379-svg/Vigrid>

Clone

Cloning is even easier if you set up a remote account

https://github.com/medvemark-10379-svg/Vigrid	Browse
Repository Type:  This is a Git repository	
G:\vigrid-wiki-angular	Browse
vigrid-wiki-angular	
Local Folder:	
[Root]	

➤ Advanced Options

Clone

SEQ ábra * ARABIC 1. ábra SourceTree

2.4. **3. Backend telepítése és indítása Visual Studio Codeban termináljában:**

2.4.1. **telepítés:** npm i

```
PS G:\vigrid-wiki-angular\vigrid-wiki-angular> npm i
npm warn deprecated prebuild-install@7.1.3: No longer
added 515 packages, and audited 516 packages in 1m
```

2.4.2. **indítás:** node src/app/server.js

```
PS G:\vigrid-wiki-angular\vigrid-wiki-angular> node src/app/server.js
Server fut: http://localhost:3000
Sikeres csatlakozás a fájlhoz: G:\vigrid-wiki-angular\vigrid-wiki-angular\src\app\Vigrid_Database.db
```

2.5. **4. Frontend telepítése és indítása Visual Studio Codeban termináljában:**

2.5.1. **telepítés:** npm i -g @angular/cli

```
PS G:\vigrid-wiki-angular\vigrid-wiki-angular> npm i -g @angular/cli
added 1 package, and changed 287 packages in 11s
```

2.5.2. **indítás:** ng serve -o

```
PS G:\vigrid-wiki-angular\vigrid-wiki-angular> ng serve -o
Initial chunk files | Names | Raw size
main.js | main | 196.02 kB |
styles.css | styles | 1.68 kB |
| Initial total | 197.70 kB

Application bundle generation complete. [1.823 seconds] - 2026-05-05T09:33:02.998Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
→ Local: http://localhost:4200/
→ press h + enter to show help
```

2.6. A BackEnd és a FrontEnd indítása Split terminálba kell történnie!

2.7. A sikeres telepítés és indítás után a weboldal elérhető lesz a böngészőben a következő

```
PS G:\vigrid-wiki-angular\vigrid-wiki-angular> npm i
up to date, audited 516 packages in 3s
120 packages are looking for funding
  run `npm fund` for details
26 vulnerabilities (1 low, 7 moderate, 18 high)
To address all issues, run:
  npm audit fix
Run `npm audit` for details.
PS G:\vigrid-wiki-angular\vigrid-wiki-angular> node src/app/server.js
Server fut: http://localhost:3000
Sikeres csatlakozás a fájlhoz: G:\vigrid-wiki-angular\vigrid-wiki-angular\src\app\Vigrid_Database.db

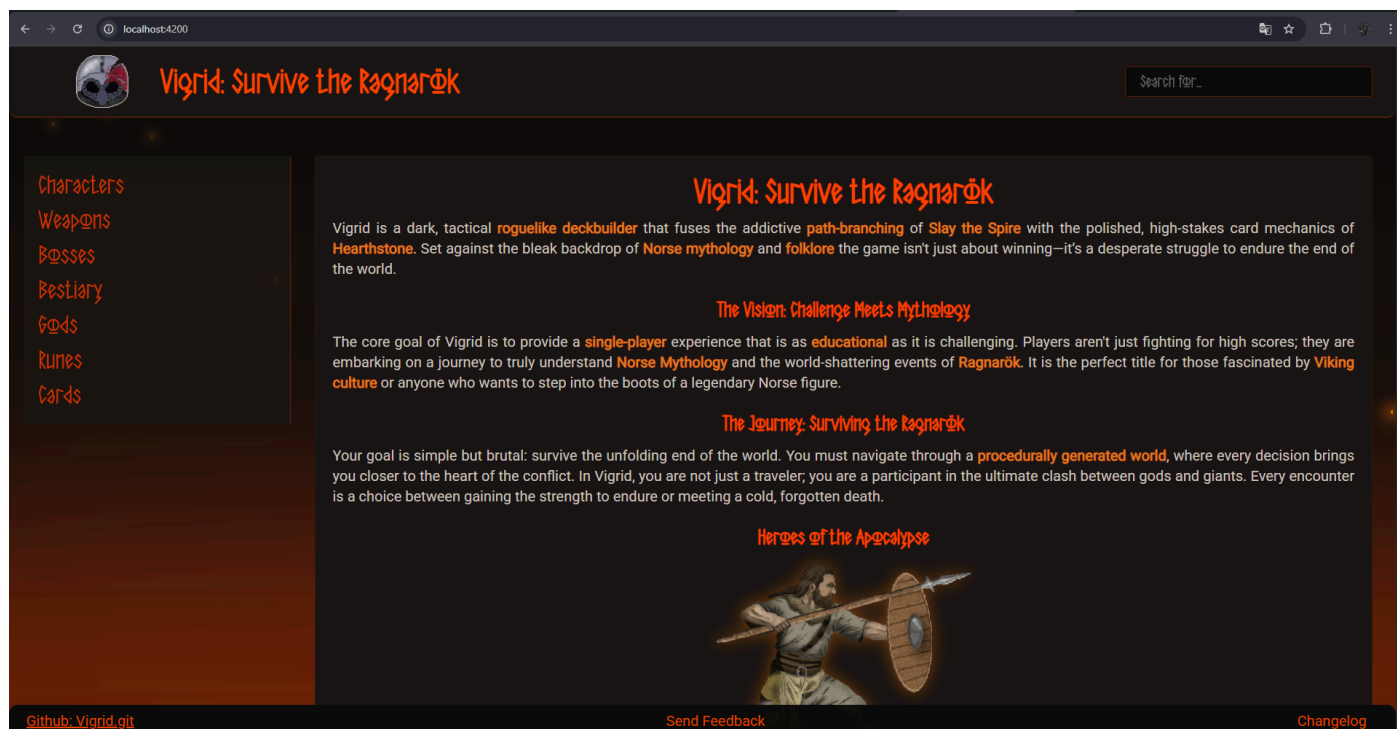
PS G:\vigrid-wiki-angular\vigrid-wiki-angular> npm i -g @angular/cli
changed 288 packages in 7s
68 packages are looking for funding
  run `npm fund` for details
PS G:\vigrid-wiki-angular\vigrid-wiki-angular> ng serve -o
Port 4200 is already in use.
Would you like to use a different port? (Y/n)
```

címe: <http://localhost:4200>

3. Weboldal funkciói:

3.1. Kezdőlap:

- 3.1.1. Lap újra töltéskor változó karakter ajánlás
- 3.1.2. Bal oldalt keresőlista
 - 3.1.2.1. Kiválasztott elemnél megjelennek a kategóriában lévő elemek
- 3.1.3. Szöveges alapú kereső
 - 3.1.3.1. Szöveg beírásakor automatikusan frissülő eredmény
 - 3.1.3.2. Nyilak és Enter használata lehetséges
- 3.1.4. Vissza a kezdőlapra, ha rákattintunk az játék ikonjára
- 3.1.5. GitHub elérési linkje
- 3.1.6. Rendszerkövetelmények megtekintése
- 3.1.7. Feedback lehetőség
- 3.1.8. Changlog nyomon követése



3.2. **Keresés:**

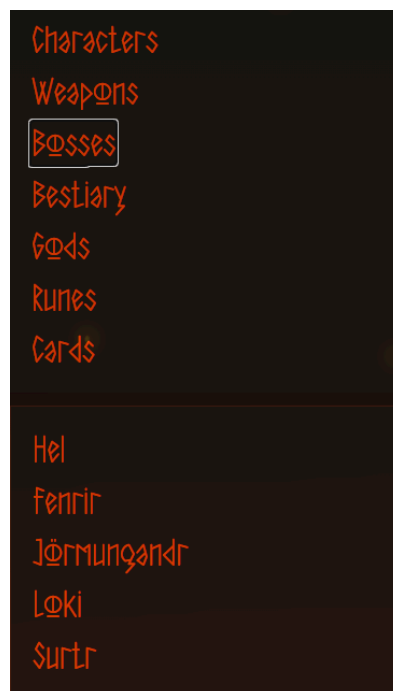
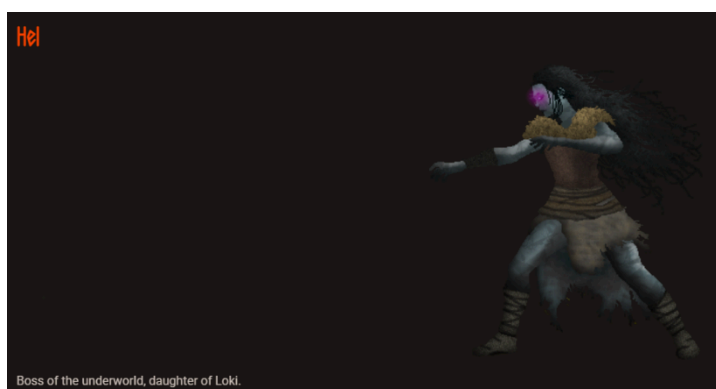
3.3. **Keresés eredménye:**

3.3.1. Név

3.3.2. Kép

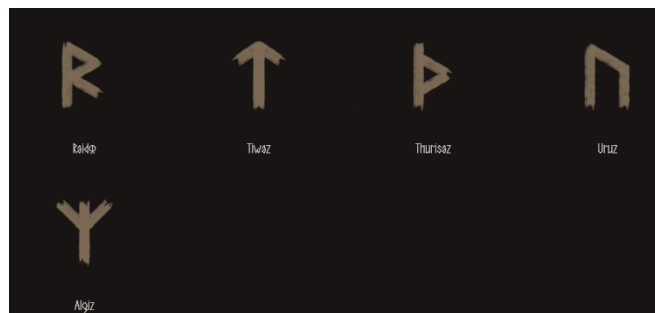
3.3.3. Leírás

3.3.4. *Statisztikája



3.4. **Keresés Kártyák vagy Rúnák esetén:**

3.4.1. Választható vizuális lista



3.5. **Keresési eredmény Kártyák esetén:**

3.5.1. + Melyik karakterrel kompatibilis az adott kártya



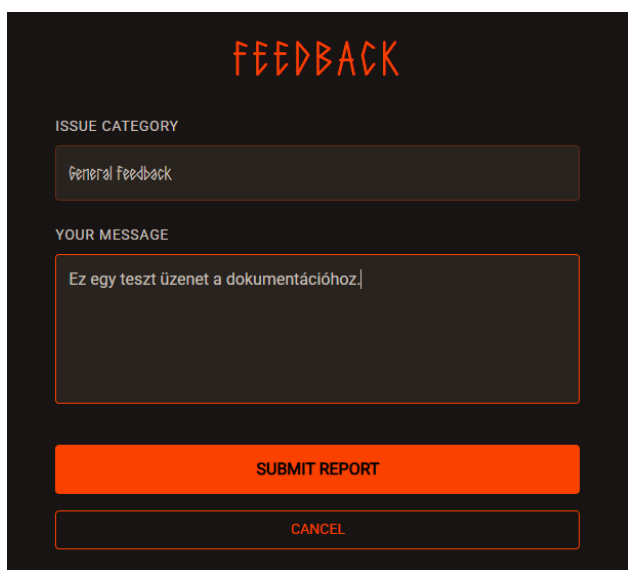
3.6. **Feedback:**

3.6.1. Kategória választás

3.6.2. Üzenet

3.6.3. Üzenet elküldése vagy mégsem gomb

3.6.4. Emailcímre megérkezik a Feedback

A dark-themed feedback form. At the top, the word "FEEDBACK" is written in a stylized orange font. Below it, the text "ISSUE CATEGORY" is followed by a dropdown menu showing "General feedback". Underneath, the text "YOUR MESSAGE" is followed by a large text area containing the placeholder text "Ez egy teszt üzenet a dokumentációhoz.". At the bottom, there are two buttons: a prominent orange "SUBMIT REPORT" button and a smaller, lighter "CANCEL" button.

New Feedback - General Feedback Inbox x



Vigrid Website Feedback <vigrid.feedback@gmail.com>
to me ▾

User report: Ez egy teszt üzenet a dokumentációhoz.

Email sent via EmailJS.com

3.7. ChangeLog:

3.7.1. Verziószám

3.7.2. Cím

3.7.3. Változtatások leírása

3.7.4. Dátum

